Assignment 6

Web Application with jQuery and Modern JavaScript Features

Objective

This assignment consists of two parts:

Part A: Develop a **Calculator Web Application with User Login** using **jQuery** and **Arrow Functions**.

Part B: Create a **Event Stopwatch with Session Logging** using **Async, Await, Promises, setInterval, and clearInterval**.

Both applications should demonstrate strong validation, modern JavaScript practices, and proper CSS styling.

Part A: Calculator with User Login

Overview

Create a two-page web application with a login page and a calculator that performs basic arithmetic operations.

Pages Required

Page 1: Login Page

Hardcoded Users: Your application should validate against pre-defined users

Form Fields:

- Email, Password, Login Button (initially disabled), Remember Me checkbox

Validation Requirements:

Email Field:

- Must be a valid email format, must end with @northeastern.edu, cannot be empty
- Show specific error: "Please enter a valid Northeastern email"

Password Field:

- Cannot be empty, minimum 8 characters, show specific error messages below the field

Validation Behavior:

- Use jQuery for all validations
- Validate on `keyup` and `blur` events
- Display error messages below each field in red
- Clear error messages when user focuses on field
- Login button remains disabled until both fields pass validation
- After validation passes, verify credentials against hardcoded users
- If credentials don't match, show error: "Invalid email or password"
- No pop-up alerts for errors

Login Process:

1. Validate form fields using jQuery
2. Check credentials against hardcoded users
3. If valid:
    - Create session in sessionStorage (or localStorage if Remember Me is checked)
    - Store: username (extract from email), email, login timestamp, isLoggedIn flag
    - Show success message with jQuery animation
    - Redirect to calculator.html after 2 seconds
4. If invalid:
    - Display error message below login button
    - Do not specify which field is incorrect

Page 2: Calculator Page

Authentication Check:

- On page load, check if user session exists in storage, if no session exists, automatically redirect to login.html

Page Layout:

Header Section:

- Display: "Welcome, [username]!", logout button

Calculator Section:

- Label: "Number 1"
- Input field 1 (for first number)
- Label: "Number 2"
- Input field 2 (for second number)
- Four operation buttons: Add, Subtract, Multiply, Divide
- Label: "Result"
- Result display field (read-only/disabled)

Validation Requirements:

Input Fields:

- Both fields required before calculation
- Only numeric values allowed (including decimals and negative numbers)
- No special characters
- Show error: "Please enter a valid number"

Operations: Check for all edge cases.

REQUIREMENT - Single Arrow Function: You must create **ONE arrow function** that handles all four operations.

Javascript -

```
const calculate = (num1, num2, operation) => {
  // operation will be: 'add', 'subtract', 'multiply', or 'divide'
  // Return the result
};
```

Do NOT create separate functions like `addNumbers()`, `subtract()`, etc.

Process:

1. Get values from input fields using jQuery selectors
2. Validate both inputs
3. If validation passes, call the single calculate arrow function with the operation type
4. Display result in the result field using jQuery
5. Use jQuery chaining for updating UI elements

Logout:

- Clear session from storage
- Show fadeOut animation using jQuery
- Redirect to login.html

CSS Requirements

- Professional, clean design
- Responsive layout (works on mobile devices)
- Error messages, Success messages
- Clear visual hierarchy
- Consistent color scheme throughout
- Proper use of Flexbox or CSS Grid for layout

Part B: Event Stopwatch with Session Logging

Overview

Create a single-page stopwatch that allows users to time events, associate them with dates, and maintain a history stored in localStorage.

Understanding the Date Picker

Important: The date picker is NOT for the timer itself. It's for organizing your timed sessions.

Purpose: Specify WHEN you're timing an activity

- Example: "Study Session on October 15, 2025"
- Example: "Workout on October 20, 2025"

This allows you to organize sessions by date and track activities over time.

Page Components

Timer Display

- Shows time in HH:MM:SS format, starts at 00:00:00
- Large, prominent display
- Read-only (user cannot edit manually)
- Updates every second when running

Event Details Section

- **Date Picker:** HTML5 date input type, defaults to today's date
- **Event Name:** Text input field with placeholder "What are you timing?"
- Both fields are disabled while timer is running
- Both fields required before starting timer

Control Buttons

- **Start:** Begins stopwatch (requires date and event name)
- **Pause/Resume:** Pauses or resumes timer (button text changes)
- **Stop & Save:** Stops timer and saves session to localStorage
- **Reset:** Clears timer to 00:00:00 without saving

Session History Section

- Displays all saved sessions from localStorage
- Each session shows:
    - Date
    - Event Name
    - Duration (formatted as HH:MM:SS)

- Most recent sessions displayed first
- Shows "No sessions recorded yet" if history is empty
- Filter by date functionality
- Statistics display: Total sessions count and total time

Validation Requirements

Use jQuery for all validations.

Date Field:
- Cannot be empty when starting timer
- Error message: "Please select a date", display below date picker in red

Event Name:
- Cannot be empty
- Minimum 3 characters, maximum 100 characters
- Only letters, numbers, spaces, hyphens, and apostrophes allowed
- Specific error messages:
    - "Event name is required"
    - "Event name must be at least 3 characters"
    - "Event name too long (max 100 characters)"
    - "Event name contains invalid characters"

Error Display:
- Show errors below respective fields in red
- Clear errors when user focuses on field
- Prevent starting timer if validation fails
- No pop-up alerts

Modern JavaScript Requirements

Use Async, Await, Promises, and Set Interval/Clear Interval for the stopwatch logic.

CSS Requirements

Required Styling:
- Large, prominent timer display (minimum 48px font size)
- Color-coded buttons
- Responsive layout (mobile-friendly)
- Professional notification styling (corner or center overlay)
- Modal dialog styling with backdrop
- History items with clear visual separation
- Error messages styled consistently
- Disabled state styling for form fields and buttons

Layout:

- Center main timer display
- Group control buttons horizontally (stack on mobile)
- Form-like layout for event details
- History section clearly separated from controls
- Statistics prominently displayed

General Requirements (Both Parts)

README.md Requirements

Each part must include a README.md with:

- Brief description of the application
- Features implemented
- Technologies used (HTML5, CSS3, JavaScript ES6+, jQuery)
- How to run the application
- Any bonus features implemented

Submission Guidelines

GitHub Repository

1. **Create Private Repository:**
    - Push both Part A and Part B to a private repository on your GitHub account using your Northeastern email account.
3. **Commit Practices:**
    - Commit regularly with meaningful messages
    - Do NOT commit after submission

Canvas Submission

1. **Create ZIP File:**
    - Name: `LastName_FirstName_Assignment6.zip`
    - Include both Part A and Part B folders
    - Include all HTML, CSS, JS, and README files
2. **Submit to Canvas:**
    - Upload ZIP file
    - In comments/remarks section, include:
        - GitHub repository URL
        - Your Northeastern email
        - Any special notes
3. **Deadline:**
    - Both Canvas and GitHub timestamps will be checked
    - Late penalty: 10% deduction per day (up to 3 days)